

实验5 图

姓名	李泽浩	学号	2025218895	实验成绩	
专业班级	软件工程 25-9 班	实验时间	2026.6.14	实验地点	新安学堂

1 实验目的

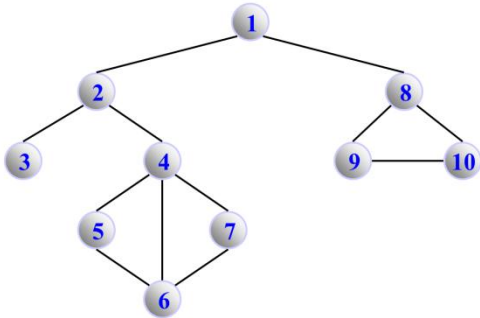
- (1) 掌握图的存储结构的设计与实现，基本运算的实现；
- (2) 掌握图的两种遍历算法、遍历生成树及遍历算法的应用；
- (3) 掌握基于图的关键路径、最短路径等实际问题求解的算法实现。

2 实验要求

- (1) 对图进行合理封装；
- (2) 在相应的存储结构上，实现基本运算；
- (3) 设计算法实现相应的实验任务；
- (4) 程序运行、测试正确；
- (5) 实验程序有较好可读性，各运算和变量的命名直观易懂，符合软件工程要求；
- (6) 程序有适当的注释，程序的书写要采用缩进格式。

3 实验任务

- <1> 设计一个无向图的类，并基于此图类实现以下功能：
- (1) 求给定图中边的个数；
 - (2) 判断该图是否是一棵树；
 - (3) 完成DFS算法，输出DFS序列；
 - (4) 完成BFS算法，输出BFS序列；



4 扩展实验

(1) 用户自定义迷宫地图（可设计和实现可视化界面），指定入口和出口，采用图相关算法寻找一条出入口之间最短路径；

5 算法设计与实现描述

库文件1-grpTypeDef.h

```
#ifndef GRPTYPEDEF_H
#define GRPTYPEDEF_H

#include <stdio>
#include <string>
#include <stdlib>
using namespace std;

#define MAXVEX 100    //最大顶点数
#define INF 65535    //无穷大
//图类型枚举
typedef enum {
    UDG, //无向图
    UDN, //无向网
    DG,  //有向图
    DN   //有向网
} GraphKind;

typedef char elementType; //顶点元素类型
typedef int eInfoType;    //边信息类型（权值）
typedef int cellType;     //邻接矩阵单元格类型
//邻接表边结点
typedef struct EdgeNode {
    int adjVer;           //邻接点编号
    eInfoType eInfo;      //边信息
    struct EdgeNode* next; //下一条边
} EdgeNode;
//邻接表顶点结点
typedef struct {
    elementType data;      //顶点数据
    EdgeNode* firstEdge;   //第一条边
} VexNode;
//图结构体（同时支持邻接矩阵和邻接表两种存储方式）
typedef struct {
    elementType Data[MAXVEX + 1]; //顶点数据（邻接矩阵用）
    cellType AdjMatrix[MAXVEX + 1][MAXVEX + 1]; //邻接矩阵
    VexNode VerList[MAXVEX + 1]; //邻接表顶点数组
    int VerNum; //顶点数
    int ArcNum; //边数
```

```

    GraphKind gKind; //图类型
} Graph;
//删除字符串左边空格
void strLTrim(char* str);
//销毁图（邻接表用）
void DestroyGraph(Graph &G);
#endif

```

库文件2-createGrpAdjLinkedList.h

```

#ifndef CREATEGRPADJLINKEDLIST_H
#define CREATEGRPADJLINKEDLIST_H

#include "grpTypeDef.h"

/*****从数据文件创建图*****/
/* 函数功能：从文本文件创建邻接表表示的图 */
/* 入口参数 char fileName[], 文件名 */
/* 出口参数: Graph &G, 创建的图 */
/* 返回值: bool, true创建成功; false创建失败 */
/* 函数名: CreateGrpFromFile1(char fileName[], Graph &G) */
/* 备注：该函数为旧版实现，直接解析邻接矩阵格式数据文件 */
/*****//

bool CreateGrpFromFile1(char fileName[], Graph &G)
{
    FILE* pFile; //定义顺序表的文件指针
    char str[1000]; //存放读出一行文本的字符串
    char strTemp[10]; //判断是否注释行
    char* ss; //字符串指针，指向fgets函数返回的字符串地址
    int i=0,j=0;
    int edgeNum=0; //边的数量
    GraphKind graphType; //图类型枚举变量
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("错误：文件%s打开失败。\\n",fileName);
        return false;
    }

    ss=fgets(str,1000,pFile);//读取第一行数据
    strncpy(strTemp,str,2);
    while((ss!=NULL) && (strstr(strTemp,"//")!=NULL) ) //跳过注释行
    {
        ss=fgets(str,1000,pFile);
        strncpy(strTemp,str,2);
    }
}

```

```
        //cout<<strTemp<<endl;
    }

    //循环结束，str中应该已经是文件标识，判断文件格式
    //cout<<str<<endl;
    if(strstr(str,"Graph")==NULL)
    {
        printf("错误：打开的文件格式错误！\n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取图的类型
    if(fgets(str,1000,pFile)==NULL)
    {
        printf("错误：读取图的类型标记失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }

    //设置图的类型
    if(strstr(str,"UDG"))
        graphType=UDG; //无向图
    else if(strstr(str,"UDN"))
        graphType=UDN; //无向网
    else if(strstr(str,"DG"))
        graphType=DG; //有向图
    else if(strstr(str,"DN"))
        graphType=DN; //有向网
    else
    {
        printf("错误：读取图的类型标记失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取顶点元素到str
    if(fgets(str,1000,pFile)==NULL)
    {
        printf("错误：读取图的顶点数据失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }

    //顶点数据放入图的顶点数组
    char* token=strtok(str," ");
    int nNum=1; //顶点数
    while(token!=NULL)
    {
```

换

```
G.VerList[nNum].data=*token;//顶点数据转换为整数，若为字符则不需转换

G.VerList[nNum].firstEdge=NULL;
//p=NULL;
//eR=G.VerList[i].firstEdge;
token = strtok( NULL, " ");
nNum++;
}

//图的邻接矩阵数据转换为邻接表
int nRow=1;    //矩阵行下标，从1开始
int nCol=1;    //矩阵列下标，从1开始
EdgeNode* eR;  //边链表尾指针
EdgeNode* p;   //边结点指针

while(fgets(str,1000,pFile)!=NULL)
{
    eR=NULL;
    p=NULL;
    nCol=1;  //列下标从1开始
    char* token=strtok(str," "); //以空格为分隔符，分割一行数据，写入邻接表
    while(token!=NULL)
    {
        if(atoi(token)>=1 && atoi(token)<INF) //有边存在
        {
            p=new EdgeNode; //创建一个邻接点边结点
            p->adjVer=nCol;   //邻接点编号为列号
            p->eInfo=atoi(token); //取得边的附加信息
            p->next=NULL;
            if(G.VerList[nRow].firstEdge==NULL)
            {
                G.VerList[nRow].firstEdge=p;
                eR=p;
            }
            else
            {
                eR->next=p;
                eR=p; //尾指针后移
            }
            edgeNum++; //边数加1
        }
        token = strtok( NULL, " "); //读取下一个子串
        nCol++; //列号加1
    }
}
```

```

        nRow++; //行号加1，读取下一行
    }
    G.VerNum=nNum; //图的顶点数
    if(graphType==UDG || graphType==UDN)
        G.ArcNum=edgeNum / 2; //无向图或网的边数等于统计的数字除2
    else
        G.ArcNum=edgeNum;
    G.gKind=graphType; //图的类型
    fclose(pFile); //关闭文件
    return true;
}

/*****从数据文件创建图*****/
/* 函数功能：从文本文件创建邻接表表示的图 */
/* 入口参数 char fileName[], 文件名 */
/* 出口参数: Graph &G, 创建的图 */
/* 返回值: bool, true创建成功; false创建失败 */
/* 函数名: CreateGraphFromFile(char fileName[], Graph &G) */
/* 备注：该函数为改进版实现，支持跳过空行和注释行 */
/*****//
bool CreateGraphFromFile(char fileName[], Graph &G)
{
    FILE* pFile; //定义顺序表的文件指针
    char str[1000]; //存放读出一行文本的字符串
    char strTemp[10]; //判断是否注释行
    int i=0,j=0;
    int edgeNum=0; //边的数量
    eInfoType eWeight; //边的信息，常为边的权值
    GraphKind graphType; //图类型枚举变量
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("错误：文件%s打开失败。\\n",fileName);
        return false;
    }

    while(fgets(str,1000,pFile)!=NULL) //跳过空行和注释行
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\\n') //空行，继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //跳过注释行
            continue;

```

```
        else    //非注释行、非空行，跳出循环
            break;
    }

    //循环结束，str中应该已经是文件标识，判断文件格式
    if(strstr(str,"Graph")==NULL)
    {
        printf("错误：打开的文件格式错误！\n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取图的类型，跳过空行
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\n') //空行，继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"/")!=NULL) //注释行，跳过，继续读取下一行
            continue;
        else //非空行，也非注释行，即图的类型标识
            break;
    }

    //设置图的类型
    if(strstr(str,"UDG"))
        graphType=UDG; //无向图
    else if(strstr(str,"UDN"))
        graphType=UDN; //无向网
    else if(strstr(str,"DG"))
        graphType=DG; //有向图
    else if(strstr(str,"DN"))
        graphType=DN; //有向网
    else
    {
        printf("错误：读取图的类型标记失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取顶点元素，到str，跳过空行
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\n') //空行，继续读取下一行
```

```

        continue;
    strncpy(strTemp, str, 2);
    if(strstr(strTemp, "//") != NULL) //注释行，跳过，继续读取下一行
        continue;
    else //非空行，也非注释行，即图的顶点元素行
        break;
}
//顶点数据放入图的顶点数组
char* token = strtok(str, " ");
int nNum = 0;
while(token != NULL)
{
    G.VerList[nNum+1].data = *token;
    G.VerList[nNum+1].firstEdge = NULL;
    //p = NULL;
    //eR = G.VerList[i].firstEdge;
    token = strtok(NULL, " ");
    nNum++;
} //顶点数为nNum，编号从1到nNum

//循环读取边的数据到邻接表
int nRow = 1; //矩阵行下标，从1开始
int nCol = 1; //矩阵列下标，从1开始
EdgeNode* eR; //边链表尾指针
EdgeNode* p;
elementType Nf, Ns; //边或弧的2个相邻顶点
while(fgets(str, 1000, pFile) != NULL)
{
    //删除字符串左边空格
    strLTrim(str);
    if (str[0] == '\n') //空行，继续读取下一行
        continue;

    strncpy(strTemp, str, 2);
    if(strstr(strTemp, "//") != NULL) //注释行，跳过，继续读取下一行
        continue;
    //nCol = 0; //列下标从0开始
    char* token = strtok(str, " "); //以空格为分隔符，分割一行数据，写入邻接表

    if(token == NULL) //分割为空串，失败退出
    {
        printf("错误：读取图的边数据失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }
}

```



```
}
Nf=*token; //获取边的第一个顶点

token = strtok( NULL, " "); //读取下一个子串，即第二个顶点
if(token==NULL) //分割为空串，失败退出
{
    printf("错误：读取图的边数据失败！\n");
    fclose(pFile); //关闭文件
    return false;
}
Ns=*token; //获取边的第二个顶点
//从第一个顶点获取行号
for(nRow=1;nRow<=nNum;nRow++)
{
    if(G.VerList[nRow].data==Nf) //从顶点列表找到第一个顶点的编号
        break;
}
//从第二个顶点获取列号
for(nCol=1;nCol<=nNum;nCol++)
{
    if(G.VerList[nCol].data==Ns) //从顶点列表找到第二个顶点的编号
        break;
}
//如果为网，读取权值
if(graphType==UDN || graphType==DN)
{
    token = strtok( NULL, " "); //读取下一个子串，即边的附加信息，常为边的权
    if(token==NULL) //分割为空串，失败退出
    {
        printf("错误：读取图的边数据失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }
    eWeight=atoi(token); //取得边的附加信息
}
eR=G.VerList[nRow].firstEdge;
while(eR!=NULL && eR->next!=NULL)
{
    eR=eR->next; //移动到链表尾，找到尾结点
}

p=new EdgeNode; //创建一个邻接点边结点
p->adjVer=nCol; //邻接点编号为列号
```

```
        if(graphType==UDN || graphType==DN) //如果为网，邻接表中对应的边设置权值，否则置为1
```

```
            p->eInfo=eWeight;
```

```
        else
```

```
            p->eInfo=1;
```

```
        p->next=NULL;
```

```
        if(G.VerList[nRow].firstEdge==NULL)
```

```
        {
```

```
            G.VerList[nRow].firstEdge=p;
```

```
            eR=p;
```

```
        }
```

```
        else
```

```
        {
```

```
            eR->next=p;
```

```
            eR=p; //尾指针后移
```

```
        }
```

```
        edgeNum++; //边数加1
```

```
    }
```

```
    G.VerNum=nNum; //图的顶点数
```

```
    if(graphType==UDG || graphType==UDN)
```

```
        G.ArcNum=edgeNum / 2; //无向图或网的边数等于统计的数字除2
```

```
    else
```

```
        G.ArcNum=edgeNum;
```

```
    G.gKind=graphType; //图的类型
```

```
    fclose(pFile); //关闭文件
```

```
    return true;
```

```
}
```

```
//销毁图
```

```
void DestroyGraph(Graph &G)
```

```
{
```

```
    EdgeNode *p,*u;
```

```
    int vID;
```

```
    for(vID=1; vID<=G.VerNum; vID++) //对每个顶点
```

```
    {
```

```
        p=G.VerList[vID].firstEdge;
```

```
        G.VerList[vID].firstEdge=NULL;
```

```
        while(p) //删除链表中的所有边结点
```

```
        {
```

```
            u=p->next; //u指向下一个结点
```

```
            delete(p); //删除当前结点
```

```
            p=u;
```

```
        }
```

```

    }
    p=NULL;
    u=NULL;
    G.VerNum=-1; //顶点数置为无效
}
#endif

```

库文件2-createGrpAdjMatrix.h

```

#ifndef CREATEGRPADJMATRIX_H
#define CREATEGRPADJMATRIX_H

#include "grpTypeDef.h"

//*****从数据文件创建图*****//
/*  函数功能： 从文本文件创建邻接矩阵表示的图          */
/*  入口参数  char fileName[], 文件名                    */
/*  出口参数： Graph &G, 创建的图                        */
/*  返回值： bool, true创建成功; false创建失败          */
/*  函数名： CreateGrpFromFile(char fileName[], Graph &G) */
//*****//
bool CreateGrpFromFile(char fileName[], Graph &G)
{
    FILE* pFile;        //定义顺序表的文件指针
    char str[1000];      //存放读出一行文本的字符串
    char strTemp[10];    //判断是否注释行
    cellType eWeight;    //边的信息，常为边的权值
    GraphKind GrpType;   //图类型枚举变量
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("错误： 文件%s打开失败。\\n",fileName);
        return false;
    }
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\\n') //空行，继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //跳过注释行
            continue;
        else //非注释行、非空行，跳出循环
            break;
    }
    //循环结束，str中应该已经是文件标识，判断文件格式

```

```
if(strstr(str,"Graph")==NULL)
{
    printf("错误: 打开的文件格式错误! \n");
    fclose(pFile); //关闭文件
    return false;
}
//读取图的类型, 跳过空行
while(fgets(str,1000,pFile)!=NULL)
{
    //删除字符串左边空格
    strLTrim(str);
    if (str[0]=='\n') //空行, 继续读取下一行
        continue;
    strncpy(strTemp,str,2);
    if(strstr(strTemp,"//")!=NULL) //注释行, 跳过, 继续读取下一行
        continue;
    else //非空行, 也非注释行, 即图的类型标识
        break;
}
//设置图的类型
if(strstr(str,"UDG"))
    GrpType=UDG; //无向图
else if(strstr(str,"UDN"))
    GrpType=UDN; //无向网
else if(strstr(str,"DG"))
    GrpType=DG; //有向图
else if(strstr(str,"DN"))
    GrpType=DN; //有向网
else
{
    printf("错误: 读取图的类型标记失败! \n");
    fclose(pFile); //关闭文件
    return false;
}
//读取顶点元素, 到str。跳过空行
while(fgets(str,1000,pFile)!=NULL)
{
    //删除字符串左边空格
    strLTrim(str);
    if (str[0]=='\n') //空行, 继续读取下一行
        continue;
    strncpy(strTemp,str,2);
    if(strstr(strTemp,"//")!=NULL) //注释行, 跳过, 继续读取下一行
        continue;
```

```

        else //非空行，也非注释行，即图的顶点元素行
            break;
    }
    //顶点数据放入图的顶点数组
    char* token=strtok(str," ");//以空格为分隔符，分割一行数据，写入图的顶点数组
    int nNum=1; //顶点数
    while(token!=NULL)
    {
        G.Data[nNum]=*token; // atoi(token);    //顶点数据转换为整数，若为字符则不需
转换
        token = strtok( NULL, " ");//继续分割下一个子串
        nNum++;
    }
    nNum--; //顶点数
    //图的邻接矩阵初始化
    int nRow=1; //矩阵行下标，从1开始
    int nCol=1; //矩阵列下标，从1开始
    if(GrpType==UDG || GrpType==DG)//如果为图，邻接矩阵中对应的边设置为1，否则设置为
INF（无穷大）
    {
        for(nRow=1;nRow<=nNum;nRow++)
            for(nCol=1;nCol<=nNum;nCol++)
                G.AdjMatrix[nRow][nCol]=0;
    }
    else
    {
        for(nRow=1;nRow<=nNum;nRow++)
            for(nCol=1;nCol<=nNum;nCol++)
                G.AdjMatrix[nRow][nCol]=INF; //INF表示无穷大
    }
    //循环读取边的数据到邻接矩阵
    int edgeNum=0; //边的数量
    elementType Nf,Ns; //边或弧的2个相邻顶点
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\n') //空行，继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //注释行，跳过，继续读取下一行
            continue;
        char* token=strtok(str," "); //以空格为分隔符，分割一行数据，写入邻接矩阵
        if(token==NULL) //分割为空串，失败退出
    }

```

```

{
    printf("错误: 读取图的边数据失败! \n");
    fclose(pFile); //关闭文件
    return false;
}
Nf=*token; //获取边的第一个顶点
token = strtok( NULL, " "); //读取下一个子串, 即第二个顶点
if(token==NULL) //分割为空串, 失败退出
{
    printf("错误: 读取图的边数据失败! \n");
    fclose(pFile); //关闭文件
    return false;
}
Ns=*token; //获取边的第二个顶点
//从第一个顶点获取行号
for(nRow=1;nRow<=nNum;nRow++)
{
    if(G.Data[nRow]==Nf) //从顶点列表找到第一个顶点的编号
        break;
}
//从第二个顶点获取列号
for(nCol=1;nCol<=nNum;nCol++)
{
    if(G.Data[nCol]==Ns) //从顶点列表找到第二个顶点的编号
        break;
}
//如果为网, 读取权值
if(GrpType==UDN || GrpType==DN)
{
    token = strtok( NULL, " "); //读取下一个子串, 即边的附加信息, 常为边的权
    if(token==NULL) //分割为空串, 失败退出
    {
        printf("错误: 读取图的边数据失败! \n");
        fclose(pFile); //关闭文件
        return false;
    }
    eWeight=atoi(token); //取得边的附加信息
}
if(GrpType==UDN || GrpType==DN) //如果为网, 邻接矩阵中对应的边设置权值, 否则
    G.AdjMatrix[nRow][nCol]=eWeight;
else
    G.AdjMatrix[nRow][nCol]=1; //atoi(token); //字符串转为整数

```

```

        edgeNum++; //边数加1
    }
    G.VerNum=nNum; //图的顶点数
    if(GrpType==UDG || GrpType==UDN)
        G.ArcNum=edgeNum / 2; //无向图或网的边数等于统计的数字除2
    else
        G.ArcNum=edgeNum; //有向图或网的边数等于统计的数字
    G.gKind=GrpType; //图的类型
    fclose(pFile); //关闭文件
    return true;
}
//删除字符串、字符数组左边空格
void strLTrim(char* str)
{
    int i,j;
    int n=0;
    n=strlen(str)+1;
    for(i=0;i<n;i++)
    {
        if(str[i]!=' ') //找到左起第一个非空格位置
            break;
    }
    //以第一个非空格字符为首字符移动字符串
    for(j=0;j<n;j++)
    {
        str[j]=str[i];
        i++;
    }
}
#endif

```

在正式实现核心算法之前，程序首先通过独立的头文件 `grpTypeDef.h`、`createGrpAdjMatrix.h` 和 `createGrpAdjLinkedList.h` 对图底层存储进行了封装。这使得系统能够灵活地基于二维数组构建邻接矩阵 `AdjMatrix`，和基于指针数组构建邻接表 `VerList`。

(1) 求给定图中边的个数

【算法思想】

借助图的邻接表特性，遍历所有顶点的边链表进行总数统计。由于在无向图（UDG）或无向网（UDN）中，同一条物理连接边会在两个端点各自的邻接链表中被重复记录一次，因此在累加所有链表节点总数后，将最终结果除以 2 即可得到图的真实边数。若是带有向属性的图结构，则直接返回遍历累计的总数即可。

【算法步骤】

- ①状态初始化：设立边数计数器 `edgeCount` 初始化为 0。
- ② 遍历节点主干：通过 `for` 循环轮询图 `G` 数组中的每一个有效顶点，获取其附属的首条边指针 `firstEdge`。
- ③ 沿链表递进：利用内部 `while` 循环沿着 `next` 指针不断向后探索，每访问一个合法的边节点即把计数器累加 1。
- ④ 最终判断：主遍历结束后，判定图的枚举类型 `gKind`，根据无向与有向的差别对累加值进行折算并返回。

【算法描述和实现】

```
#include "grpTypeDef.h"
#include "createGrpAdjMatrix.h"
#include "createGrpAdjLinkedList.h"
#include <iostream>
using namespace std;
//用邻接表存储的图G，计算边的数目
int CountEdgesFromAdjList(Graph &G)
{
    int edgeCount = 0; // 边数计数器
    for (int i=0;i<G.VerNum;i++)
    {
        EdgeNode* p = G.VerList[i+1].firstEdge; // 获取第i个顶点的第一条边
        while (p != NULL)
        {
            edgeCount++; // 统计边数
            p = p->next; // 移动到下一条边
        }
    }
    if (G.gKind == UDG || G.gKind == UDN){ // 如果是无向图或无向网，边数需要除以2
        return edgeCount /= 2;
    }
    else { // 有向图或有向网，边数直接统计
        return edgeCount;
    }
}
```

(2) 判断该图是否是一棵树

【算法思想】

图论中对“树”的严格定义是“无环且全连通”的无向图。借助 DFS 对分支进行探测，在递归深入时，若当前准备访问的相邻节点已经被访问过，且该相邻节点并非引导我们来到此处的“父节点”，即证明探测到了交叉路径，构成了环结构。当 DFS 搜索返回后，若判定图全域无环，且所有的有效顶点都被成功打上了访问标记即无孤立连通分量，则证明此图是一棵合法的树。

【算法步骤】

① 初始化：创建全局 `visited` 数组并清零，从 1 号顶点发起首次检测，将其父节点编号参数设定为 -1。

② 递归判断：进入 `iftree` 后首先将当前点设为已访问。遍历所有邻接分支，若遇未访问节点则将其作为子节点继续递归深入；若邻接节点已访问且编号排除了父节点带来的合法回环可能，则立即向外层抛出 `false`。

③ 连通复检：获取递归结果后，再次遍历 `visited` 数组，若发现任何未被触及的孤立节点，则覆盖判定结果为不是树。

【算法描述和实现】

```
// v当前访问的顶点编号
// parent当前访问顶点的父节点编号
// visited[]访问标记数组
bool iftree(Graph &G, int v,int parent,int visited[])
{
    visited[v]=1;
    EdgeNode* p = G.VerList[v].firstEdge; // 获取第v个顶点的第一条边
    while (p != NULL)
    {
        if(visited[p->adjVer]==0)
        {
            if (!iftree(G, p->adjVer, v, visited)) // 子树有环，则整图不是树
                return false;
        }
        else if(p->adjVer!=parent)// 如果访问过的邻接点不是父节点，说明有环
        {
            return false;
        }
    }
}
```

```
        p = p->next; // 移动到下一条边
    }
    return true;
}
```

(3) 完成 DFS 算法，输出序列

【算法思想】

DFS 算法的推演逻辑依赖于递归。当算法触达一个顶点时，第一时间执行数据标记与处理工作。紧接着寻找该起点的第一个处于未被访问状态的邻接点，以此进行同样深度的垂直探索。当某条连通路径触底时，代码逻辑跟随递归特性层层回溯，转入其他未解的平行分支。

【算法步骤】

① 遍历记录：函数执行伊始，标记当前传入顶点 v 已被征用，并将其内置的大写字母通过 ASCII 偏移量换算为直观的整型序号进行打印（如 $\text{data} - 'A' + 1$ ）。

② 深层遍历：抓取依附于该起点的边链表头指针，开启 while 循环侦测每一个直接连通的靶点。

③ 进行回溯：检查靶点的全局访问标志，一旦发现为 0，即刻回溯发起新一轮 DFS 调用进行递归。

【算法描述和实现】

```
//邻接表存储的图的DFS算法实现
void DFS(Graph &G, int v, int visited[])
{
    visited[v] = 1; // 标记当前顶点已访问
    // 输出映射后的编号 (A=1, B=2, ..., J=10)
    cout << (G.VerList[v].data - 'A' + 1) << " ";

    EdgeNode* p = G.VerList[v].firstEdge; // 获取第v个顶点的第一条边
    while (p != NULL)
    {
```

```

        if (visited[p->adjVer] == 0) // 如果邻接点未访问过
        {
            DFS(G, p->adjVer, visited); // 递归访问邻接点
        }
        p = p->next; // 移动到下一条边
    }
}

//邻接矩阵存储的图的DFS算法实现
void DFS2(Graph &G, int v, int visited[])
{
    visited[v] = 1; // 标记当前顶点已访问
    // 输出映射后的编号 (A=1, B=2, ..., J=10)
    cout << (G.Data[v] - 'A' + 1) << " ";
    for (int i = 1; i <= G.VerNum; i++)
    {
        if (G.AdjMatrix[v][i] != 0 && visited[i] == 0) // 存在边且未访问
        {
            DFS2(G, i, visited); // 递归访问邻接点
        }
    }
}
}

```

(4) 完成 BFS 算法，输出序列

【算法思想】

广度优先搜索依赖于先入先出机制的队列来维持层级扩散。它从原点出发将其入队，只要队列维持运转，就出队首个节点，并将其所有处于未知状态的邻接节点压入队尾储备。由于先处理完毕当前步数距离的所有节点再开启更远层的遍历，这种横向扫描适用于无权图的平铺探测。

【算法步骤】

① 初始化：声明基于 `int` 类型的 STL 队列容器 `q`，强行注入起点序号，并在 `visited` 数组上打上占位标识。

② 出队输出：在 `while` 的循环下，只要容器内部有残留目标，就提取并弹出队列，完成数据输出映射操作。

③ 分支收拢：遍历由刚刚弹出的元素伸出的所有边链，将其中未被标记的周边顶点统一标记以此避免重复入列，并成批次推送至队列末端待命。

【算法描述和实现】

```
//邻接表存储的图BFS算法实现
void BFS(Graph &G, int v, int visited[])
{
    queue<int> q;           // BFS 辅助队列
    visited[v] = 1;        // 标记起点已访问
    q.push(v);             // 起点入队

    while (!q.empty())
    {
        int cur = q.front(); // 取出队首顶点
        q.pop();
        // 输出映射后的编号 (A=1, B=2, ..., J=10)
        cout << (G.VerList[cur].data - 'A' + 1) << " ";
        EdgeNode* p = G.VerList[cur].firstEdge;
        while (p != NULL)
        {
            if (visited[p->adjVer] == 0)
            {
                visited[p->adjVer] = 1; // 入队时就标记，防止重复入队
                q.push(p->adjVer);
            }
            p = p->next;
        }
    }
}

//临界矩阵存储的图BFS算法实现
void BFS2(Graph &G, int v, int visited[])
{
    queue<int> q;           // BFS 辅助队列
    visited[v] = 1;        // 标记起点已访问
    q.push(v);             // 起点入队

    while (!q.empty())
    {
        int cur = q.front(); // 取出队首顶点
        q.pop();
        // 输出映射后的编号 (A=1, B=2, ..., J=10)
        cout << (G.Data[cur] - 'A' + 1) << " ";
        for (int i = 1; i <= G.VerNum; i++)
```

```
    {
        if (G.AdjMatrix[cur][i] != 0 && visited[i] == 0) // 存在边且未访问
        {
            visited[i] = 1; // 入队时就标记，防止重复入队
            q.push(i);
        }
    }
}
```

(5) 扩展实验：用户自定义迷宫地图求最短路径

【算法思想】

将包含多重障碍的二维矩阵空间建模为隐式的网格连通图，利用 BFS 独有的“第一到达即为最优解”距离特性规划路径。封装复合了二维坐标、步数权重以及用于回溯的 parent 指针的结构体节点。每当从前驱坐标扩散至四个可用方位时，就将子节点与前驱节点绑定。抵达终点后，借助前置引用进行倒推，还原并刻画行走轨迹。

【算法步骤】

① 初始化：利用方向数组 directions 遍历四个方向，通过 isValid 过滤函数挡下诸如越墙、出界和折返等非法衍生行为。

② dfs：构建并初始化包含复合状态节点的探索队列，推入包含起点信息的首发侦测节点，打上 visited 拦截网。

③ 指针寻底：不断抛出父级节点，派生通过校验的合法坐标，压入队尾。当到达终点时截断循环，并携带当前节点向外部抛出成果。

④ 打印路径：激活 printPath，通过提取结果内部包含的指针流不断倒拉复原，在矩阵对应单元格注入特殊的 8 标记，之后输出通道。

【算法描述和实现】

```
#include <iostream>
```

```

#include <vector>

#include <queue>

using namespace std;

// 迷宫的行数和列数
const int ROW = 10;
const int COL = 10;

// 定义一个点结构体
struct Point {
    int x, y;

    Point(int _x, int _y) : x(_x), y(_y) {}
};

// 定义一个方向数组，表示上下左右四个方向
int directions[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

bool isValid(int map[ROW][COL], int x, int y, bool visited[ROW][COL]) {
    // 是否可以访问点 (x, y)，需要满足以下条件：

    if (x < 0 || x >= ROW || y < 0 || y >= COL) // 越界
        return false;

    if (map[x][y] == 1) // 墙壁
        return false;

    if (visited[x][y]) // 已访问过
        return false;

    return true;
}

// 定义一个节点结构体，用于 BFS 搜索，包含当前节点位置、距离和父节点指针
struct node {
    node* parent; // 父节点指针

    Point pos;    // 当前节点位置

    int dist;     // 从起点到当前节点的距离

    node(Point _pos, int _dist, node* _parent) : pos(_pos), dist(_dist), parent(_parent) {}
};

// BFS 搜索迷宫的最短路径
node searchmin(int map[ROW][COL], Point start, Point end) {
    bool visited[ROW][COL] = {false}; // 访问标记数组

    queue<node*> q; // BFS 辅助队列，存储当前节点指针

    q.push(new node(start, 0, nullptr)); // 起点入队

    visited[start.x][start.y] = true; // 标记起点已访问

    while (!q.empty()) {
        auto current = q.front();

        q.pop();

        Point pt = current->pos; // 当前节点位置

        int dist = current->dist;

        if (pt.x == end.x && pt.y == end.y) { // 到达终点
            return *current; // 返回当前节点，包含路径信息
        }
    }
}

```

```

    }

    // 遍历四个方向
    for (int i = 0; i < 4; i++) {
        int newX = pt.x + directions[i][0];
        int newY = pt.y + directions[i][1];
        if (isValid(map, newX, newY, visited)) {
            visited[newX][newY] = true; // 标记新点已访问
            q.push(new node(Point(newX, newY), dist + 1, current)); // 新点入队，路径长度加1
        }
    }
}

return node(end, -1, nullptr); // 无法到达终点
}

void printPath(node* n, int map[ROW][COL]) {
    if (n == nullptr)
        return;

    map[n->pos.x][n->pos.y] = 8; // 标记路径点
    printPath(n->parent, map); // 递归打印父节点路径
}

int main() {
    // 0: 通道, 1: 墙壁
    int map[ROW][COL] = {
        {0, 0, 1, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 1, 0, 1, 1, 1, 1, 0, 0},
        {0, 0, 0, 0, 1, 0, 0, 0, 0, 0},
        {0, 1, 1, 1, 1, 0, 1, 1, 1, 0},
        {0, 0, 0, 0, 0, 0, 1, 0, 0, 0},
        {0, 1, 1, 0, 1, 1, 1, 0, 1, 0},
        {0, 0, 1, 0, 0, 0, 1, 0, 1, 0},
        {1, 0, 1, 1, 1, 0, 1, 0, 1, 0},
        {0, 0, 0, 0, 1, 0, 0, 0, 1, 0},
        {0, 1, 1, 0, 0, 0, 1, 0, 0, 0}
    };

    Point start(0, 0); // 起点 (0,0)
    Point end(9, 9);   // 终点 (9,9)

    cout << "==== 迷宫最短路径搜索 (BFS) =====> << endl;

    node result = searchmin(map, start, end);

    if (result.dist != -1) {
        cout << "找到最短路径! " << endl;

        printPath(&result, map); // 打印路径

        for (int i = 0; i < ROW; i++) {
            for (int j = 0; j < COL; j++) {
                if (i == start.x && j == start.y) cout << "S "; // 起点
            }
        }
    }
}

```

```

        else if (i == end.x && j == end.y) cout << "E ";    // 终点
        else if (map[i][j] == 8) cout << "* ";            // 路径
        else if (map[i][j] == 1) cout << "■ ";            // 墙壁
        else cout << ". ";                                  // 空地

    }

    cout << endl;

}

} else {

    cout << "没有路径可达终点!" << endl;

}

return 0;

}

```

6运行结果截图及说明

(1) 计算边数功能测试

```

===== 邻接表存储的图--求边（弧）数目 =====
图类型：无向图(UDG)
边（弧）数目（邻接表遍历计算）：12

```

说明：程序成功通过给定的文件构建出无向图模型。经过算法遍历整个邻接表并精确折算无向边的重复计算后，准确向控制台输出了该测试图中独立的物理边总数量，无多算与漏算的情况。

(2) 树形结构判定测试

```

===== 邻接表存储的图--判断是否树 =====
图不是树！

```

说明：程序从系统配置的首发节点发起了带有历史节点溯源标记的深度探索。判定系统确认该连通分量中不存在违背逻辑的回环连线，且图内所有的活动顶点均在此次探测中被全面标记，最终输出“图不是树！”的确切结论。

(3) DFS 遍历测试

```

图类型：无向图(UDG)
1 2 3 4 5 6 7 8 9 10

```

说明：在以核心节点 1 为起点的深探任务中，算法展现出了明显的一路到底特质。根据内部边链表映射到字母组合的排布逻辑，程序按照“先深后宽”的基准策略打印出有序的路线队列。

(4) BFS 遍历测试

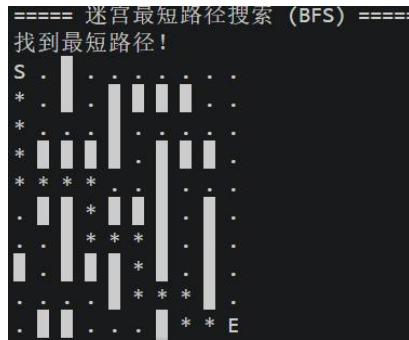
```

图类型：无向图(UDG)
1 2 8 3 4 9 10 5 6 7

```

说明：引入外部队列引擎完成后，控制台打印出平整规律的遍历痕迹。相较于前一项测试，此时节点被极其严格地按照“相邻距离层次”以批次状平铺抛出，完全契合 BFS 既定的推演逻辑。

(5) 扩展寻路挑战测试



说明：面对一张内部包含密集实体墙的 10x10 自定义封锁网络，探寻算法在寻获通关要道后触发了反向描边逻辑。控制台将矩阵复原，并通过 * 标识打点出了从入口 S 到终点 E 之间完美规避死胡同的最优躲避通道，呈现效果极具层次与直观性。

7总结、心得和建议

【总结】

本次实验围绕图的存储结构与核心遍历算法展开了实验。在底层结构设计上，程序通过独立的头文件成功对图进行了合理封装，构建了涵盖邻接矩阵与邻接表的存储方式。在算法层面，实验实现了图中边数统计、基于无环连通特性的树形结构判定等基础运算，还编写了深度优先搜索（DFS）与广度优先搜索（BFS）算法。同时，在扩展实验环节，程序将复杂的二维迷宫空间抽象为网格连通图，利用 BFS 队列引擎配合前置节点指针，成功求解并输出了起点至终点的最短路径通道。

【心得】

（1）标记走过的路是防止死循环的关键：

这次用 DFS 遍历复杂的图，我最大的感受就是必须严密控制每个顶点的访问标记。无向图的边是双向的，如果没有拦住回头的路，程序瞬间就会在两个点之间来回跳，陷入死循环。这让我想起之前用 C++ 写回溯算法测试时，发现依然会漏掉一些结果，都是因为状态没控制好。这次加上了对 parent（父节点）的精准判断，成功防住了死循环，感觉自己以后写复杂逻辑时的“防坑”意识更强了。

（2）数据结构可以作为解决复杂问题的“工具”：

以前学顺序队列的时候，觉得它只是个“先进先出”的简单数组，但这次的迷宫寻路扩展实验完全刷新了我的认知。把封装好的队列直接当作工具，使用 BFS 像石子落水一样一圈圈向外扩散寻找最短路径，效果特别直观。这种一层层向外层探索的算法逻辑让我很有启发。

（3）遍历特性的差异化应用场景：

通过控制台输出的 DFS 序列与 BFS 序列，我更为深刻地理解了不同遍历策略在真实工程中的选型依据。DFS 的“一路到底”特性天然契合对拓扑结构的深层挖掘，例如在判断图是否为一棵树的任务中，依靠递归深入并配合父节点

(parent) 参数校验，能够极其敏锐地侦测到不合法的逻辑回环。

相对而言，BFS 结合队列的“水波纹式”扩散机制，可以解决无权图最短距离问题。在自定义迷宫地图的求解中，正是利用了 BFS “首达即最优”的距离特性，才得以筛选出规避墙壁的最短路径。这印证了算法工具的挑选必须与题目吻合。

【建议】

(1) **增加大规模数据的测试：**这次实验我们用的图比较小，只有十几个顶点，程序跑起来很轻松。但如果以后遇到成千上万个顶点的数据，DFS 里面太深的递归可能会导致系统栈溢出，BFS 用的队列也会占用特别大的内存。建议以后可以试着用更庞大的数据来测试程序的极限，或者学着用非递归比如自己手动建一个 stack 的方法来写 DFS，这样代码在极端情况下会更稳定。

(2) **结合可视化工具辅助调试：**现在程序只能在黑底白字的控制台里输出一堆数字和字母的遍历序列，凭空脑补图的结构有点困难，检查代码连线对不对时也特别容易眼花。建议平时做图论实验时，可以借助一些简单的画图小工具，或者用纸画一下，把代码里的顶点和边变成真正看得见的网状图。这样哪里连错了直接一目了然，找 bug 会轻松很多，也能帮我们更好地理解图的结构。